# React + System Design Interview Q&A;

## Q1: Difference between useEffect and useLayoutEffect

**Answer:**
- useEffect runs asynchronously after the paint. It does not block the UI.
- useLayoutEffect runs synchronously after all DOM mutations but before paint, so it can block rendering.
- Use useEffect for API calls, subscriptions, logging, etc.
- Use useLayoutEffect when you need to measure DOM elements or synchronously mutate layout before paint.

```
import React, { useEffect, useLayoutEffect, useRef } from "react";

function Example() {
  const boxRef = useRef();

  useLayoutEffect(() => {
    console.log("useLayoutEffect: Box width", boxRef.current.offsetWidth);
  }, []);

  useEffect(() => {
    console.log("useEffect: Runs after paint");
  }, []);

  return <div ref={boxRef} style={{ width: "200px", height: "100px", background: "skyblue" }}>Box</di
}
```

## Q2: Handling 10,000+ Products in an E-Commerce App

**Answer:**
Challenges: Rendering large lists can hurt performance.
**Techniques:**
1. Virtualized Lists → only render visible items.
2. Keys → unique keys help React update efficiently.
3. Lazy Loading & Infinite Scroll.
4. Caching → avoid refetching data.
5. Scalability → backend caching (Redis), CDN for static assets, horizontal scaling.

```
import { FixedSizeList as List } from "react-window";

function ProductList({ products }) {
  return (
    return (
      <List
        height={500}
        itemCount={products.length}
        itemSize={100}
        width={400}
      >
        {({ index, style }) => (
          <div style={style}>
            {products[index].name}
          </div>
        )}
      </List>
    );
}
```

## Q3: Explain React Portals and their Use Cases

**Answer:**
Portals let you render children into a different part of the DOM tree, outside the parent hierarchy.
**Use cases:** Modals, tooltips, dropdowns.

```
import ReactDOM from "react-dom";

function Modal({ children }) {
  return ReactDOM.createPortal(
    <div className="modal">{children}</div>,
    document.getElementById("modal-root")
  );
}
```

## Q4: Designing a Scalable Chat Application (React + System Design)

**Frontend (React):**
- Reusable components (ChatBox, MessageList, Input).
- Performance optimizations → React.memo, useCallback, lazy loading.
- Virtualized messages (don’t render thousands at once).

**Backend:**
- WebSockets for real-time communication.
- Microservices for users, messages, notifications.
- Load Balancers for distributing requests.
- Horizontal scaling (add servers).
- Caching (Redis).
- Security with encryption.

```
function ChatMessages({ messages }) {
  return (
    <div className="chat-container">
      {messages.map(msg => (
        <div key={msg.id} className="chat-message">
          <strong>{msg.sender}:</strong> {msg.text}
        </div>
      ))}
    </div>
  );
}
```